
TryRAG

Architektur-Review & arc42-Dokumentationsgerüst

Abgleich gegen die Vorlesungs-/Prüfungsanforderungen

Prüft Übung für Übung, was die Vorlesung verlangt, gegen den aktuellen Stand (Präsentation *TryRAG-Architektur* + Repository) und liefert anschließend ein ausfüllbares arc42-Gerüst für die Abgabe.

Modul	Softwarearchitektur (Prof. N. Weeger), SS 2026
Team	Constantin Dendtel • Juri Sindel • Dominic Bechtold
Abgabe-Deadline	02.07.2026 (GitLab-Repo, arc42-konform)
Prüfungstag	06.07.2026 (online, 7,5 min/Person + Diskussion)

June 28, 2026

Contents

Teil I – Gap-Analyse	2
1 Worauf der Dozent bewertet	2
2 Gesamtbefund	2
3 Artefakt-Mapping: Übung → arc42 → Status	2
4 Bewertung je Übung + konkrete To-dos	4
5 Priorisierte Lücken vor der Abgabe	5
6 Hinweise für die mündliche Prüfung	6
Teil II – arc42-Dokumentationsgerüst	7
7 Einführung und Ziele (arc42 §1)	7
8 Randbedingungen (arc42 §2)	7
9 Kontextabgrenzung (arc42 §3)	7
10 Lösungsstrategie (arc42 §4)	7
11 Bausteinsicht (arc42 §5)	7
12 Laufzeitsicht (arc42 §6)	8
13 Verteilungssicht (arc42 §7)	8
14 Querschnittliche Konzepte (arc42 §8)	8
15 Architekturentscheidungen (arc42 §9)	8
16 Qualitätsanforderungen (arc42 §10)	8
17 Risiken und technische Schulden (arc42 §11)	8
18 Glossar (arc42 §12)	9

Teil I – Gap-Analyse

1 Worauf der Dozent bewertet

Aus den Folien (Prüfung, Infos zur Prüfung, Übungen 1–6, arc42-Mapping auf Folie 36) lassen sich die Bewertungskriterien klar ableiten:

1. **Vollständigkeit:** Die Abgabe muss *mindestens* alle in den Übungen definierten Artefakte enthalten. Die Summe der Übungen ergibt eine vollständige, **arc42-konforme** Dokumentation im Repo.
2. **Nachvollziehbarkeit der Entscheidungen:** Nicht nur *was*, sondern *warum* – inkl. **Alternativen** und **Entscheidungsfindung**. Das ist der explizit genannte Kern („Sie zeigen, dass Sie verstanden haben, wie Architektur funktioniert“).
3. **Qualitätsgetriebenheit:** Requirements + NFRs treiben die Architektur; jede Entscheidung begünstigt nachweisbar bestimmte Qualitätsszenarien (Trade-offs benennen). „Architektur ist weder korrekt noch inkorrekt ohne Qualitätsanforderungen.“
4. **Messbarkeit:** Qualitätsszenarien nach dem Bass-Schema (Source, Stimulus, Artifact, Environment, Response, **Response Measure**).
5. **Sichten:** C4 (Context/Container/Component/Dynamic) bzw. die Bass-Viewtypes; Architektur auf mehreren Ebenen erklären.
6. **Dokumentationsqualität:** korrekt, aktuell, wartbar, verständlich, angemessen – „lebt im Repository“ (versioniert, diffbar).

2 Gesamtbefund

Die Präsentation ist **inhaltlich stark** und deckt die geforderten Themen breit ab – oft über das Pflichtmaß hinaus (9 ADRs statt 6, DDD, Defense-in-Depth, Traceability-Idee). Der **kritische Punkt** ist nicht der Inhalt, sondern der **Abgabe-Träger**:

*Die Abgabe ist das **GitLab-Repository**, nicht die Präsentation. In diesem Code-Repository (**Chat-Assistant-AI**) liegen die in der Präsentation referenzierten arc42-Artefakte aktuell **nicht** vor.*

Konkret fehlen hier (oder liegen in einem separaten Repo – unbedingt verifizieren): `functional_requirements`, `constraints.md`, `qualities.md`, `/scenarios/`, `quality-tree.md`, `glossary.md`, `traceability.md`, `risks.md`, die 9 Architektur-ADRs sowie `adrs/patterns/design_patterns.md`. Vorhanden sind nur ältere, komponentenbezogene ADRs (`01_frontend_chatbot` ... `008_transform_to_microservices`) und `docs/doc.tex` (AP-1A-Implementierungsdoku – nicht die Architekturabgabe).

Wichtigste Sofortmaßnahme. Sicherstellen, dass das Abgabe-Repo genau die Datei-/Ordnerstruktur von Folie 36 hat und alle referenzierten Dateien real existieren. Die Präsentation darf keine Datei nennen, die der Prüfer im Repo nicht findet.

3 Artefakt-Mapping: Übung → arc42 → Status

Legende: ✓ **vorhanden** = in der Präsentation belegt △ **Repo prüfen** = referenziert, Datei im Repo verifizieren × **fehlt** = nicht erkennbar / anzulegen.

Übung	Gefordertes Artefakt	arc42-Sektion	Status
Ü1	Readme.md (App-Beschreibung + Gruppen-Infos)	§1 Intro & Goals	△ Repo prüfen
Ü1	functional_requirements.md (7–10 FRs, priorisiert)	§1	✓ vorhanden (11 FRs, MoSCoW)
Ü1	constraints.md (Annahmen + Randbed.)	§2 Constraints	✓ vorhanden (Inhalt da)
Ü1	use-case-diagram	§3 Context & Scope	✓ vorhanden (Auszug)
Ü2	qualities.md (5+1 ISO-25010, begründet)	§10 Quality	✓ vorhanden (Tabelle)
Ü2	/scenarios/ (≥2 je Top-3, messbar)	§10	△ Repo prüfen (IDs genannt)
Ü2	quality-tree.md (Utility Tree)	§10	△ Repo prüfen
Ü3	ADR Applikationstyp	§9 Decisions	✓ vorhanden (ADR 002)
Ü3	ADR Programmiersprache(n)	§9	✓ vorhanden (ADR 003.2)
Ü3	ADR Framework	§9	✓ vorhanden (ADR 003.1)
Ü3	2 weitere ADRs	§9	✓ vorhanden (004–009)
Ü3	glossary.md (Ubiquitous Language)	§12 Glossary	△ Repo prüfen
Ü3	bounded-contexts (boxes & lines)	§5 Building Block	✓ vorhanden
Ü4	ADR Architekturpattern(s) (vs. Szenarien, auf Domänen)	§9	△ Repo prüfen (impliz. in 001)
Ü4	ADR Design Patterns (3 Kernstellen)	§9	✓ vorhanden (Adapter/CB/PubSub)
Ü5	C4 System Context	§3	✓ vorhanden (L1)
Ü5	C4 Container (BCs + Technik)	§5	✓ vorhanden (L2)
Ü5	C4 Component	§5	✓ vorhanden (L3)
Ü5	C4 Dynamic (≥2 Use-Cases)	§6 Runtime View	✓ vorhanden (FR-1, FR-2)
Ü6	ADR Schnittstellen	§9	✓ vorhanden (ADR 006)

Übung	Gefordertes Artefakt	arc42-Sektion	Status
Ü6	risks.md (≥3 Risiken, Tech-Debt, Mitigation)	§11 Risks	✓ vorhanden (6 Risiken)
<i>arc42-Sektionen, die die Übungen nicht erzwingen, aber das Template (Folie 87) listet:</i>			
–	Solution Strategy	§4	× fehlt (implizit)
–	Deployment (K8s/EU)	View §7	× fehlt (kein Diagramm)
–	Crosscutting Concepts	§8	△ Repo prüfen (Security/Resilienz da)

4 Bewertung je Übung + konkrete To-dos

Übung 1 – Anforderungen

Stark: 11 FRs mit MoSCoW-Priorisierung, rollenbasiert; Use-Case-Diagramm; Randbedingungen klar (EU-Hosting, DSGVO/AVV, Tenant-Isolation, 99,9 % SLA, PII-Filter, self-hosted ChromaDB).

- **[TODO: Readme]** Gruppen-Infos je Mitglied ergänzen: **Name, Matrikelnummer, Hochschule, E-Mail** (explizit gefordert) + kurze App-Beschreibung.
- FRs als *User Stories* im Format „Als ... möchte ich ..., um ...“ formulieren (Folie 40) – schafft Anschluss an die Use-Cases.
- **constraints.md**: Annahmen *und* Randbedingungen klar trennen.

Übung 2 – Softwarequalität

Stark: ISO-25010-Tabelle mit Prioritäten; 5 „Hoch“ + 2 „Niedrig“; messbare Ziele (TTFT ≤ 1,5 s, Retrieval ≤ 250 ms); Architecture Drivers benannt (S2, S3, Z3).

- **Pflicht-Detail:** Jedes Szenario in /scenarios/ im **vollständigen Bass-6-Tupel** (Source, Stimulus, Artifact, Environment, Response, **Response Measure**). In der Präsentation sind bisher nur IDs (Z1–Z4, S1–S3, E1–E3) sichtbar – im Repo müssen die ausformulierten, *messbaren* Szenarien liegen (≥2 je Top-3-Attribut).
- **qualities.md**: für die 3 wichtigsten Attribute je einen **Use-Case** angeben, in dem das Attribut am wichtigsten ist (Ü2.1.3), und je Attribut begründen, *warum* wichtig/unwichtig.
- Trade-offs explizit machen (Folie 50): z. B. PII-Filter (Sicherheit) kostet Latenz (Effizienz).

Übung 3 – Designen

Stark: 9 ADRs, DDD mit 7 Bounded Contexts + Shared Kernel, Ubiquitous Language adressiert.

- △ **Repo prüfen** glossary.md sicherstellen (Ubiquitous Language als echte Begriffsliste: Tenant, Workspace, RAG, Chunk, Embedding, Retrieval, MCP, Cost-Attribution ...).
- ADR-Nummerierung dokumentieren: Die Übung erwartet ADR 001 Applikationstyp / 002 Sprache / 003 Framework. Euer Schema weicht ab (001 Stil, 002 Anwendungstyp, 003.1/003.2). Das ist legitim, aber in **traceability.md** eine **Mapping-Tabelle** „Übungsanforderung → ADR-Nummer“ aufnehmen, damit der Prüfer jedes Pflicht-ADR findet.

- Jedes ADR im Standard-Template (Title, Status, Context, Decision, Consequences inkl. *negativer*) – Folie 64.

Übung 4 – Patterns

Stark: Design Patterns an 3 Kernstellen (Adapter, Circuit Breaker, Publish/Subscribe) mit Szenario-Zuordnung; weitere genannt (Repository, Facade, Token-Bucket).

- **Architekturpattern** als eigenes ADR schärfen: Der Stil (DDD-Microservices, ADR 001) ist da, aber Ü4 verlangt die Pattern-Wahl *explizit gegen Qualitätsszenarien* begründet *und* auf die Bounded Contexts *abgebildet*. Tabelle: Pattern → unterstütztes Szenario → betroffene Domäne.
- Begründung „welche Szenarien wie gut unterstützt werden“ ausformulieren (nicht nur ID-Liste).

Übung 5 – Dokumentation

Stark / vollständig: C4 L1–L3 + 2 Sequenzdiagramme (FR-1, FR-2).

- Diagramme als **Diagrams-as-Code** (PlantUML/C4-PlantUML oder Structurizr) im Repo ablegen – der Dozent betont versionierbar/diffbar (Folien 94–99). `.drawio` ist laut Folie 36 ok, aber Code ist die ausdrücklich empfohlene Kür.
- Container-Diagramm: explizit als „Bounded Contexts + technische Sicht“ kennzeichnen (Ü5.2-Wortlaut).

Übung 6 – Schnittstellen + Reflexion

Stark: Schnittstellen-Mix pro Kante begründet (REST/gRPC/Async/MCP/SDK); `risks.md` mit 6 Risiken + Mitigation + Tech-Debt.

- Schnittstellen-Auswahl an die Synchron-/Asynchron-/Echtzeit-Taxonomie der Vorlesung (Folien 106–108) anknüpfen – zeigt bewusste Auswahl.
- `risks.md`: zu jedem Risiko die Frage „wo entsteht Tech-Debt beim *Wachstum*“ getrennt von der Mitigation beantworten (Ü6.2).

5 Priorisierte Lücken vor der Abgabe

#	Lücke / Maßnahme	Priorität
1	Abgabe-Repo gem. Folie 36 anlegen; alle referenzierten <code>.md</code> -Dateien real einchecken	kritisch
2	<code>/scenarios/</code> als vollständige, messbare Bass-Szenarien ausformulieren	kritisch
3	<code>traceability.md</code> mit Mapping FR/NFR → Baustein → Szenario → ADR (und Übung → ADR-Nr.)	hoch
4	<code>Readme.md</code> mit Gruppen-Infos (Name/Matrikelnr./HS/E-Mail)	hoch
5	<code>glossary.md</code> (Ubiquitous Language)	hoch
6	Architekturpattern-ADR: Pattern → Szenario → Domäne explizit	hoch
7	arc42 §4 Solution Strategy (1 Seite) ergänzen	mittel
8	arc42 §7 Deployment-Diagramm (K8s, eu-central-1)	mittel
9	Diagramme als PlantUML/C4-Code versionieren	mittel

6 Hinweise für die mündliche Prüfung

- **Format:** alle online, jede Gruppe präsentiert, **7,5 min pro Person** + Diskussion, alle gleich viel Arbeit. Die drei Bereiche fair aufteilen (z. B. Anforderungen/Qualität | Entscheidungen/Patterns | Sichten/Schnittstellen/Risiken).
- **Roter Faden:** App-Idee → Requirements/NFRs → warum diese Architektur → welche NFRs werden wie begünstigt → Alternativen & Entscheidungsfindung. Genau diese Kette verlangt Folie 7.
- **Auf Alternativ-Fragen vorbereiten:** Zu jedem großen ADR den verworfenen Weg *und den Ausschlaggrund* parat haben (Monolith, Serverless, ESB; direkte SDKs vs. Abstraktion; pgvector/Qdrant statt Chroma).
- **Trade-offs aktiv ansprechen** statt sie zu verstecken (Distributed-Systems-Komplexität, Eventual Consistency, Halluzinationen).

Teil II – arc42-Dokumentationsgerüst

Vorlage zum Ausfüllen/Einchecken (eine Datei je Sektion bzw. gem. Folie 36). Inhalte sind aus eurer Präsentation vorbefüllt; **[TODO: ...]** markiert, was noch aus dem Team kommen muss.

7 Einführung und Ziele (arc42 §1)

Aufgabe. TryRAG ist eine multi-mandantenfähige B2B-SaaS-RAG-Plattform: isolierter Workspace pro Tenant, RAG-Chatbot mit Quellenangaben, Agenten für Mehrschritt-Tasks via MCP, plus Verwaltung (RBAC, Usage-Monitoring, Key-Management), DSGVO-konform und EU-gehostet.

Top-Qualitätsziele. AZ-1 Tenant-Isolation & Datenschutz • AZ-2 Zuverlässigkeit (99,9 % SLA) • AZ-3 schnelle, korrekte Antworten.

Funktionale Anforderungen (Auszug, MoSCoW). MUST: FR-1 RAG-Frage, FR-2 Upload, FR-3 Login, FR-8 RBAC, FR-10 API-Keys. SHOULD: FR-4 Agenten/MCP, FR-6 Kalender, FR-9 MCP-Anbindung, FR-11 Usage. COULD: FR-5 Bot-Parameter, FR-7 Abteilungs-Termine. **[TODO: volle FR-Liste FR-1..FR-11 als User Stories]**

Stakeholder. Endnutzer, Tenant-Admin, Plattform-Betreiber, Datenschutz/Compliance.

8 Randbedingungen (arc42 §2)

EU-Hosting (eu-central-1, Frankfurt) • DSGVO-konform inkl. AVV mit Cloud-/LLM-Providern • strikte Tenant-Isolation • 99,9 % SLA der Plattform-API • PII-Filter vor jedem Provider-Versand • self-hosted Vektor-DB (ChromaDB). **[TODO: Annahmen vs. Randbedingungen trennen; org./technische/zeitliche Constraints]**

9 Kontextabgrenzung (arc42 §3)

Fachlicher Kontext / System Context (C4 L1). TryRAG kapselt RAG, LLM-Routing, Billing und Tenant-Isolation. Externe Akteure: Mitarbeiter, Tenant-Admin. Externe Systeme: LLM-Provider (OpenAI/Anthropic), MCP-Zielsysteme (Kalender/Ticketing), Identity Provider (OIDC/OAuth2). Personenbezogene Daten verlassen das System nur PII-gefiltert. (*Diagramm: C4-Context, Präsentation Folie 8.*)

10 Lösungsstrategie (arc42 §4)

[TODO: 1 Seite]: Kurzbegründung der Kernweichen je Qualitätsziel: DDD-Microservices (Isolation/Skalierung) • Polyglot Node.js/Python (passende Tech je Workload) • LLM-Abstraktion (Provider-Unabhängigkeit, Cost-Tracking) • self-hosted ChromaDB + RLS (DSGVO/Isolation) • Event-Bus (Entkopplung Hot-Path).

11 Bausteinsicht (arc42 §5)

Bounded Contexts: Auth, Tenant, LLM-Routing, RAG, Agent, Billing, Audit + Shared Kernel (UserProfile, TenantId, i18n, CloudEvents). **Container (C4 L2):** SPA (React/TS), API-Gateway/BFF (Node.js), Redis, RAG-Service (Python), LLM-Routing (Python), Agent-Service (Python), Auth/Tenant/Billing/Audit (Node.js), ChromaDB, PostgreSQL (RLS), Object Storage (S3 EU), Event-Bus (RabbitMQ). **Komponenten (C4 L3):** RAG-Context (Retriever, Context

Builder, Ingestion, Embedding) + LLM-Routing (Provider-Interface, OpenAI-/Anthropic-Adapter, PII-Filter, Cost-Tracker). (*Folien 9 – 11.*)

12 Laufzeitsicht (arc42 §6)

FR-1 RAG-Query-Flow: SPA → Gateway (JWT/Rate-Limit) → Retriever → ChromaDB (Top-K) → LLM-Routing (PII-gefiltert) → Provider → SSE-Stream; danach asynchron `llm.request.completed` → Billing/Audit. **FR-2 Upload-Flow:** Gateway (JWT/RBAC/Quota) → Ingestion → Object Storage; ab 202 Accepted asynchron Parsing/Chunking/Embedding → ChromaDB; Abschluss `document.indexed`. (*Folien 16 – 17.*)

13 Verteilungssicht (arc42 §7)

[TODO: Deployment-Diagramm]: Kubernetes in eu-central-1; Services als Deployments + HPA; PostgreSQL/ChromaDB/Object-Storage/RabbitMQ/Redis als Stateful-/Managed- Komponenten; Edge mit TLS + WAF.

14 Querschnittliche Konzepte (arc42 §8)

Security (Defense in Depth): TLS+WAF → OIDC + kurzlebige JWTs → RBAC → PostgreSQL Row-Level-Security → ChromaDB-Collection-Konvention `tenant_{id}_{type}` → Cross-Tenant-Tests im CI; PII-Filter Richtung Provider. **Resilienz:** Circuit Breaker + Fallback-Provider, Retry/Backoff, Service-Isolation, Async-Entkopplung, Caching (Redis), HPA. **Observability:** OpenTelemetry + Prometheus/Grafana/Loki/Tempo (ADR 008). (*Folie 18.*)

15 Architekturentscheidungen (arc42 §9)

9 ADRs (je Kontext/Entscheidung/Begründung/Alternativen/Konsequenzen): 001 DDD-Microservices • 002 Cloud-native Multi-Tenant SaaS • 003.1 React/TS-SPA • 003.2 Polyglot Node.js+Python • 004 ChromaDB self-hosted • 005 LLM-Adapter + Cost-Tracking • 006 Schnittstellen-Mix • 007 RabbitMQ • 008 Observability • 009 OIDC/OAuth2 + JWT + RBAC + API-Keys. **Patterns:** Adapter, Circuit Breaker, Publish/Subscribe (3 Kernstellen) + Repository, Facade/Gateway, Token-Bucket. **[TODO: Architekturpattern-ADR: Tabelle Pattern → Szenario → Domäne]**

16 Qualitätsanforderungen (arc42 §10)

ISO-25010-Priorisierung: Hoch – Zuverlässigkeit, Sicherheit, Funktionale Eignung, Benutzbarkeit, Effizienz; Mittel – Kompatibilität; Niedrig – Wartbarkeit, Übertragbarkeit. Architecture Drivers: S2 Cross-Tenant, S3 PII, Z3 Provider-Fallback. **Utility Tree + Szenarien [TODO: vollständige Bass-6-Tupel je Szenario, ≥ 2 je Top-3-Attribut, mit Response Measure (z. B. $TTFT \leq 1,5$ s; $Retrieval \leq 250$ ms; $Recovery$; $Cross-Tenant-Zugriff = 0$)].**

17 Risiken und technische Schulden (arc42 §11)

Tenant-Isolation-Leak (niedrig/katastrophal – RLS+Collection+CI) • LLM-Provider-Dependency (mittel/hoch – Abstraktion+Fallback+CB) • Last/Skalierung (hoch/spürbar – Queue+HPA+Cache) • VectorDB-Bulking (Retention+Splitting+Re-Indexing) • Halluzinationen (Quellen+Confidence-Threshold) • Datenschutz/PII (PII-Filter+EU+AVV). **[TODO: Tech-Debt beim Wachstum je Risiko getrennt benennen.]**

18 Glossar (arc42 §12)

[TODO: Ubiquitous Language als Begriffsliste]: Tenant, Workspace, RAG, Chunk, Embedding, Retrieval (ANN), Bounded Context, Shared Kernel, MCP, PII-Filter, Cost-Attribution, Circuit Breaker, RLS ...